Proposal for Final Project

Statistical Programming Paradigms and Workflows (BSPH 140.777)

Team: The Masons

1. Title and team

Working project title: Choosing a Hospital: A Patient-Facing Shiny App for Comparing Condition-Specific Outcomes.

Team: The Masons.
Team members: Sankalp (Sunny) Kumar, Wanting Shan, Shiyi Zhang, Yutong (Benny) Li, Mason Cleveland
Team leader: Sankalp Kumar

2. Question and audience

Our guiding question is: **How can patients and caregivers compare hospitals for a selected condition using publicly available measures of mortality, readmissions, cost and overall quality?**

Choosing a suitable hospital involves interpreting information that can be difficult to find or compare. A hospital's overall rating does not necessarily describe its performance for a particular disease and condition. Patients also encounter missing measures, different reporting periods, and historical financial information without understanding how these affect a comparison.

We propose a Shiny application for patients and caregivers who are exploring options for planned care or discussing hospital choices with a clinician. Users will select a condition and location, inspect available hospital outcomes, and compare measures alongside their definitions and reporting periods. The initial version will focus on Maryland hospitals and three conditions: heart attack, heart failure, and pneumonia.

Our motivation is to bridge the gap between available information and usable information, allow patients to exercise their rights to choose health care services. The app will support specifically informed comparisons rather than estimate an individual's disease risk or identify a universally best hospital. For now, we will not account for specialist availability, insurance networks, or payment details.

3. Related work

Medicare Care Compare already provides a public interface for comparing healthcare providers, including hospitals. The CMS Provider Data Catalog supplies the underlying public datasets that can support reproducible analyses and alternative interfaces.

Zocdoc offers another approach to helping patients find care. Its Baltimore provider listings bring together location, accepted insurance, patient reviews, and appointment availability, allowing users to search for providers and book appointments.

Our project will build on these resources rather than introduce a new source of hospital quality measurement. Its intended contribution is a focused, condition-first comparison workflow for a small set of outcomes. We will review Care Compare's existing hospital interface to inform our development.

CMS documentation also informs interpretation. Its complications and deaths guidance explains the meaning of mortality measures, while its unplanned hospital visits guidance distinguishes readmissions from other return-visit outcomes. We will use these definitions in our work.

4. Data and access

Sources and available measures

We will acquire data through the **CMS Provider Data Catalog API**, which allows public requests without an API key. This API uses a different endpoint structure from the general data.cms.gov API. Four dataset endpoints have already been accessed successfully:

Dataset	Dataset ID	Intended use
Hospital General Information	`xubh-q36u`	Hospital identity, location, type, and overall stars
Complications and Deaths - Hospital	`ynj2-r877`	Condition-specific mortality and complication measures
Unplanned Hospital Visits - Hospital	`632h-zaca`	Condition-specific readmissions and other unplanned visits
Historical Payment and Value of Care - Hospital	`c7us-v4mf`	Optional, explicitly historical Medicare payment context

Hospital information can be linked to outcomes using the CMS facility ID.

Collection and preparation workflow

We will use reusable R functions to send requests, retrieve successive pages using `limit` and `offset`, and retry transient failures. We may downloaded row counts with API totals. The preliminary workflow uses pages of 500 records. We might retain collection metadata so that the app can report a source and retrieval date.

We will filter outcome tables to the intended measure IDs and join records by facility ID and condition. We will retain original score strings and footnotes while creating numeric fields for analysis. Missing or suppressed scores will remain missing rather than being converted to zero.

Access and interpretation limitations

Public access does not guarantee uninterrupted service. We might cache a dated extract for the deployed app and provide code to refresh it, rather than make an API call for every user interaction. We will document measurement periods and download dates.

5. Planned work

We will organize development around the following tasks:

1. **Confirm the initial measures.**
2. **Build a reproducible data pipeline.**
3. **Develop transparent comparisons.**
4. **Build the Shiny interface.**
5. **Verify and deploy the product.**

We will begin with measure-by-measure comparisons rather than a composite ranking.

6. Programming paradigms

We plan to integrate **functional programming**. In addition, we will use **command-line programming** or **machine learning**

Functional programming will structure acquisition and transformation. A reusable API function will be mapped over named dataset identifiers using `purrr::imap()`. Similar functions will select measures, parse scores, and summarize availability across conditions. This approach makes the workflow consistent across datasets and reduces repeated code. The preliminary R acquisition script already uses this pattern.

Command-line programming will provide a reproducible workflow from acquisition to reporting. We may extend the existing shell entry point into a script that accepts a state

argument, runs the R data pipeline, performs validation, and renders the Quarto data report. The script will stop when a required stage fails and use documented relative paths. This gives collaborators one entry point for refreshing the app's cached data and checking the resulting outputs.

The components will work together as follows: a command-line entry point runs functional R acquisition and cleaning routines, which create validated data for Shiny and a rendered Quarto report. We will discuss whether the planned command-line component is sufficiently substantive at the proposal meeting.

Machine learning is not necessary for the proposed question, but will potentially play a role in this project, depending on time, data quality, and feasibility. We still need to decide which ML technique (decision trees, KNN, logistic regression, etc) is appropriate for this project.

7. Software

The main implementation will use **R**:

- `httr2` for API requests, response handling, and retries;
- `dplyr`, `tidyr`, and `readr` for preparation and CSV input/output;
- `purrr` for mapping reusable functions over datasets and conditions;
- `ggplot2` for descriptive comparison graphics;
- `shiny` for the interactive application;
- `knitr` and Quarto for executable documentation and PDF reports.

A Bash script will coordinate the command-line workflow. Git and a shared GitHub repository will support collaboration, review, and contribution tracking. We plan to deploy through shinyapps.io, subject to account access and hosting limits, and will provide local run instructions as well. We will record package versions and document dependencies, potentially with .renv.

8. Product

This is tentative, but we envision that the final product will be a deployed, patient-facing **Shiny hospital comparison app**, accompanied by a shared repository containing acquisition code, preparation functions, documentation, and the final Quarto report.

A user will select a condition and then narrow hospitals by available location fields, such as state or city. The app will display condition-specific mortality and readmission measures alongside hospital characteristics and overall stars. Users will be able to sort the table and select hospitals for a closer comparison. Definitions, missing-value explanations, and measurement dates will be accessible within the interface.

The repository will document how to install dependencies, refresh data, and start the app using `shiny::runApp()`. The deployed URL will be added once hosting is established. Cached data

will keep the interface responsive and allow it to function independently of temporary API outages.

9. Timeline and feasibility

Period	Planned milestone
November 11–13	Finalize the proposal, confirm scope, and retain evidence of API access
November 16–20	Meet with instructor or TA and revise the plan based on feedback
November 21–30	Complete acquisition, cleaning, provenance, and data validation
December 1–10	Build the Shiny comparison interface and explanatory graphics
December 11–16	Check usability and correctness, deploy, and prepare documentation and slides
December 17	Present the project in class

The exact deadlines for slides, the final write-up, and repository submission will be confirmed on CoursePlus.

We envision a manageable initial version as a join being produced by a small script. For the final app, we plan to consolidate the core preparation workflow in R to simplify execution and maintenance.

The **minimum viable product** is a Maryland app with three condition choices, a hospital comparison table, a selected-hospital comparison view, and clear definitions, dates, and missingness indicators. We have demonstrated API access, so the main remaining challenges are interpretation, measure alignment, usability, and reliable deployment.

We will manage scope by prioritizing the three-condition comparison. If data completeness limits a comparison, we will show that limitation rather than impute a hospital's performance. If hosting presents difficulties, local execution will remain available while we resolve deployment. Further extensions will be considered only after the core workflow is working and reproducible.

10. Team Responsibilities

- Sankalp (Sunny) Kumar: Confirm the initial measures.
- Wanting Shan: Build a reproducible data pipeline.
- Shiyi Zhang: Develop transparent comparisons.

- Mason Cleveland: Build the Shiny interface.
- Yutong (Benny) Li: Verify and deploy the product.

We will coordinate through regular meetings and use GitHub to share code, track progress, and collaborate on the project.